RAPPORT DE TÂCHE

Mission 6 — Tâche 1 : Gérer les tests

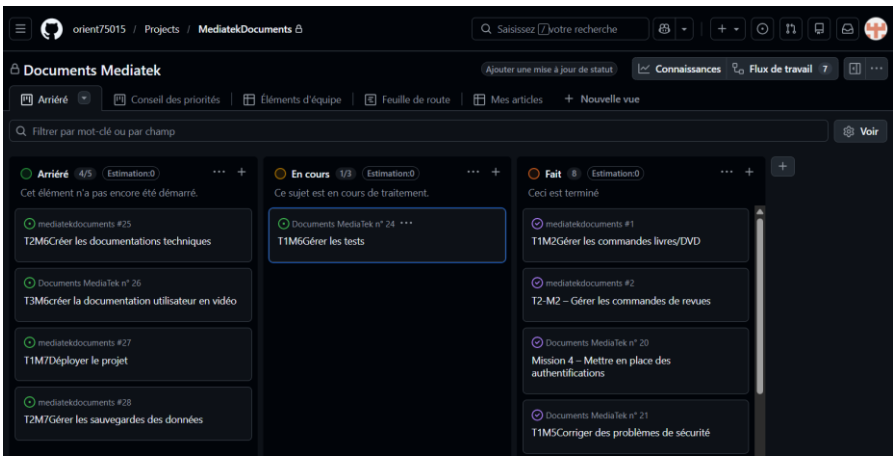
Projet MediatekDocuments — C# / Postman

Projet	MediatekDocuments — BTS SIO SLAM
Mission	Mission 6 — Tests et documentation
Tâche	Tâche 1 — Gérer les tests
Temps estimé	5 heures
Temps réel	5 heures
Outils utilisés	Visual Studio (MSTest), Postman
Fichiers modifiés	MediaTekDocuments.Tests\UnitTest1.cs

1. Demande de la tâche

- Écrire les tests unitaires sur les classes du package Model (en plus du test unitaire ParutionDansAbonnement écrit précédemment)
- Construire une collection de tests dans Postman pour contrôler les fonctionnalités de l'API d'accès à la BDD
- Rédiger un plan de tests comportant l'ensemble des tests effectués par catégorie

2. Kanban



Capture du Kanban GitHub — M6T1 dans la colonne 'In Progress'

3. Temps estimé et temps réel

Indicateur	Valeur
Temps estimé	5 heures
Temps réel	5 heures (installation Postman incluse)
Écart	Aucun

4. Tests unitaires C#

4.1 Classes testées

Les tests unitaires ont été écrits sur toutes les classes du package model, en plus du test ParutionDansAbonnement déjà existant :

Classe testée	Tests effectués
ParutionDansAbonnement	5 tests : date dans intervalle, avant, après, égale début, égale fin
Livre	1 test : constructeur et propriétés correctement initialisées
Dvd	1 test : constructeur et propriétés correctement initialisées
Revue	1 test : constructeur et propriétés correctement initialisées
Exemplaire	1 test : constructeur et propriétés correctement initialisées
CommandeDocument	1 test : constructeur et propriétés correctement initialisées
CommandeRevue	1 test : constructeur et propriétés correctement initialisées
Utilisateur	1 test : constructeur et propriétés correctement initialisées
Service	2 tests : constructeur + méthode ToString
Suivi	2 tests : constructeur + méthode ToString

4.2 Exemple de code — test unitaire

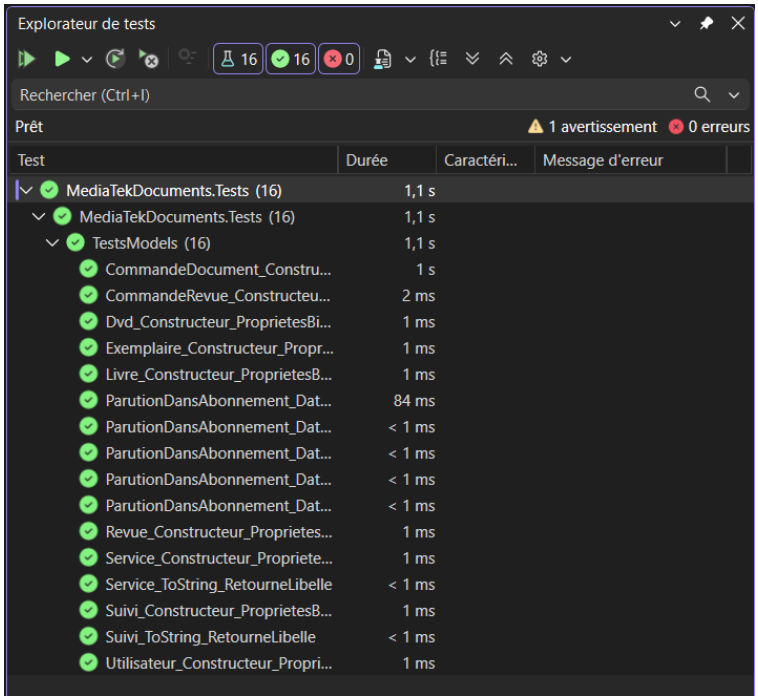
Exemple du test sur la classe Livre :

```
[TestMethod]
public void Livre_Constructeur_ProprietesBienInitialisees()
{
    Livre livre = new Livre("00017", "Catastrophes au Brésil", "",
        "", "Philippe Masson", "", "10014", "Policier",
        "00004", "Ados", "JN002", "Jeunesse romans");
    Assert.AreEqual("00017", livre.Id);
    Assert.AreEqual("Philippe Masson", livre.Auteur);
    Assert.AreEqual("Catastrophes au Brésil", livre.Titre);
}
```

```
55
56      #region Tests Livre
57      [TestMethod]
        0 références | 0 modification | 0 auteur, 0 modification
58      public void Livre_Constructeur_ProprietesBienInitialisees()
59      {
60          Livre livre = new Livre("00017", "Catastrophes au Brésil", "", "", "Philippe Masson", "",
61              "10014", "Policier", "00004", "Ados", "JN002", "Jeunesse romans");
62          Assert.AreEqual("00017", livre.Id);
63          Assert.AreEqual("Philippe Masson", livre.Auteur);
64          Assert.AreEqual("Catastrophes au Brésil", livre.Titre);
65          Assert.AreEqual("Policier", livre.Genre);
66          Assert.AreEqual("Ados", livre.Public);
67          Assert.AreEqual("Jeunesse romans", livre.Rayon);
68      }
69      #endregion
```

Capture de UnitTest1.cs — exemple de test unitaire sur Livre (MediaTekDocuments.Tests\UnitTest1.cs)

4.3 Résultat des tests unitaires

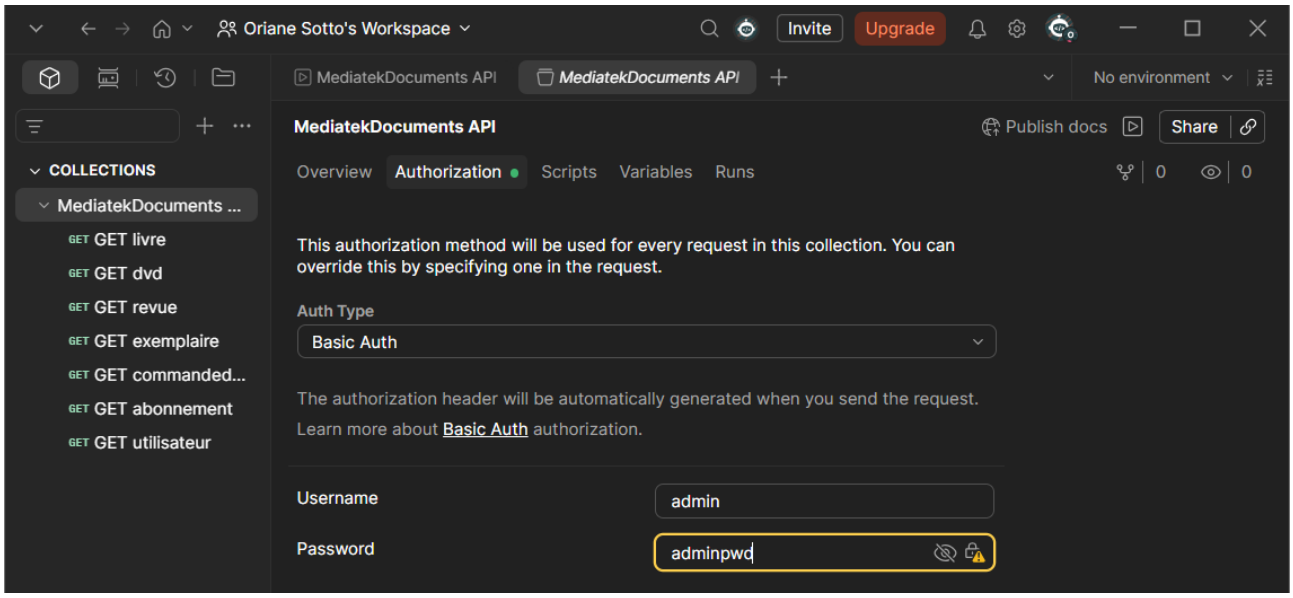


Capture de l'Explorateur de tests Visual Studio — 16 tests sur 16 réussis (vert)

5. Tests fonctionnels Postman

5.1 Installation et configuration

Postman a été installé et configuré pour tester l'API REST de MediatekDocuments. Une collection 'MediatekDocuments API' a été créée avec une authentification Basic Auth (admin/adminpwd) configurée au niveau de la collection, héritée par toutes les requêtes.



Capture de Postman — collection MediatekDocuments API avec Basic Auth configurée

5.2 Requêtes testées

7 requêtes GET ont été créées et testées dans la collection :

Requête	URL
GET livre	http://localhost/rest_mediatekdocuments/livre
GET dvd	http://localhost/rest_mediatekdocuments/dvd
GET revue	http://localhost/rest_mediatekdocuments/revue
GET exemplaire	http://localhost/rest_mediatekdocuments/exemplaire
GET commandedocument	http://localhost/rest_mediatekdocuments/commandedocument
GET abonnement	http://localhost/rest_mediatekdocuments/abonnement
GET utilisateur	http://localhost/rest_mediatekdocuments/utilisateur

Note : les routes exemplaire, commandedocument et abonnement nécessitent un paramètre id pour retourner des résultats. Sans paramètre, l'API retourne une liste vide avec un code 200. Les tests vérifient uniquement que l'API répond correctement.

5.3 Exemple de code — test Postman

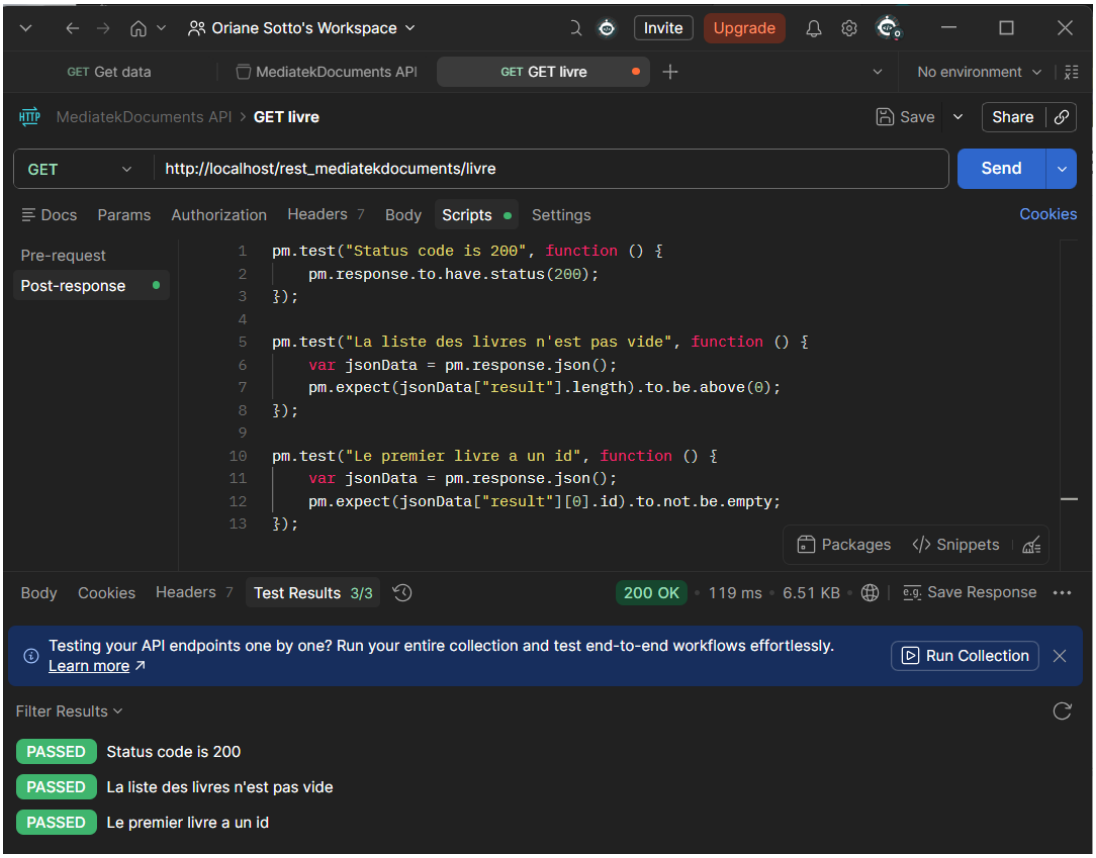
Exemple du script de test pour la requête GET livre :

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("La liste des livres n'est pas vide", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData["result"].length).to.be.above(0);
});

pm.test("Le premier livre a un id", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData["result"][0].id).to.not.be.empty;
});
```

URL : http://localhost/rest_mediatekdocuments/livre — 3 tests : status 200, liste non vide, premier id non vide.



Capture Postman — GET livre avec 3/3 tests PASSED

5.4 Choix des routes testées

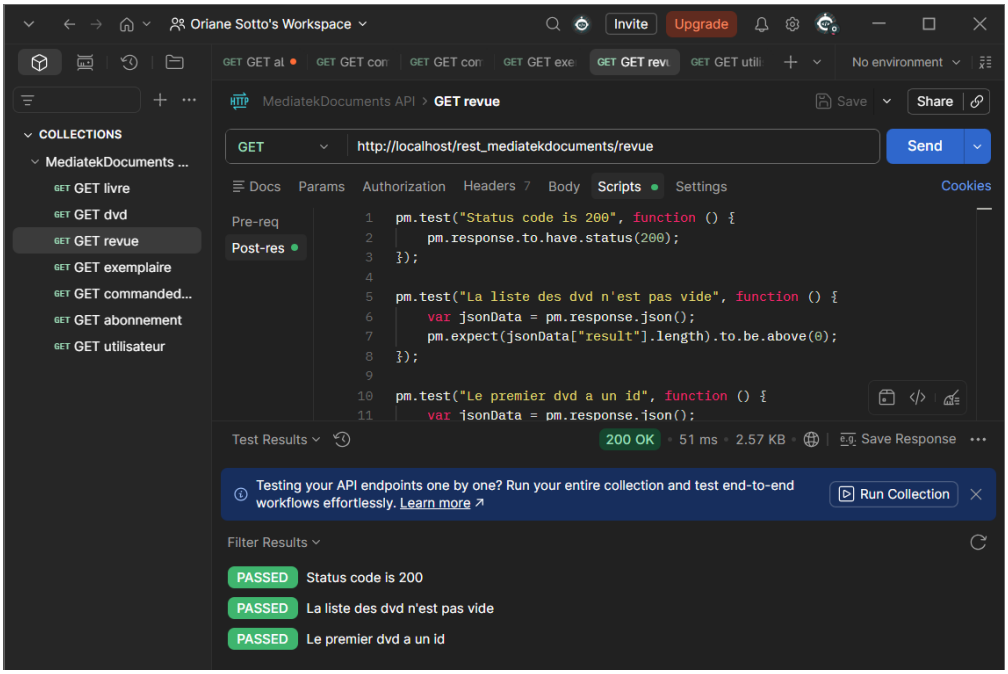
7 routes ont été sélectionnées pour couvrir l'ensemble des fonctionnalités de l'API :

- livre, dvd, revue : routes retournant une liste complète sans paramètre
- exemplaire, commandedocument, abonnement : routes nécessitant un paramètre id — sans paramètre l'API retourne une liste vide avec un code 200
- utilisateur : route nécessitant un paramètre login

Les routes suivi, genre, rayon et public n'ont pas été testées séparément car elles fonctionnent sur le même principe que livre/dvd/revue.

5.5 GET dvd

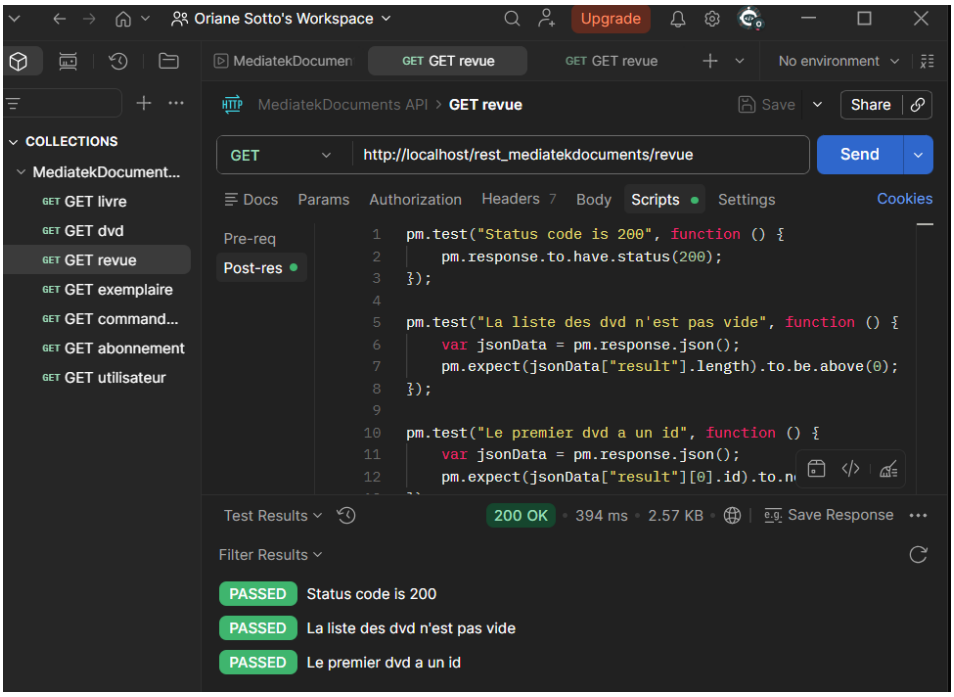
URL : http://localhost/rest_mediatekdocuments/dvd — mêmes 3 tests que livre, adaptés aux DVD.



Capture Postman — GET dvd : 3/3 PASSED

5.6 GET revue

URL : http://localhost/rest_mediatekdocuments/revue — mêmes 3 tests que livre, adaptés aux revues.

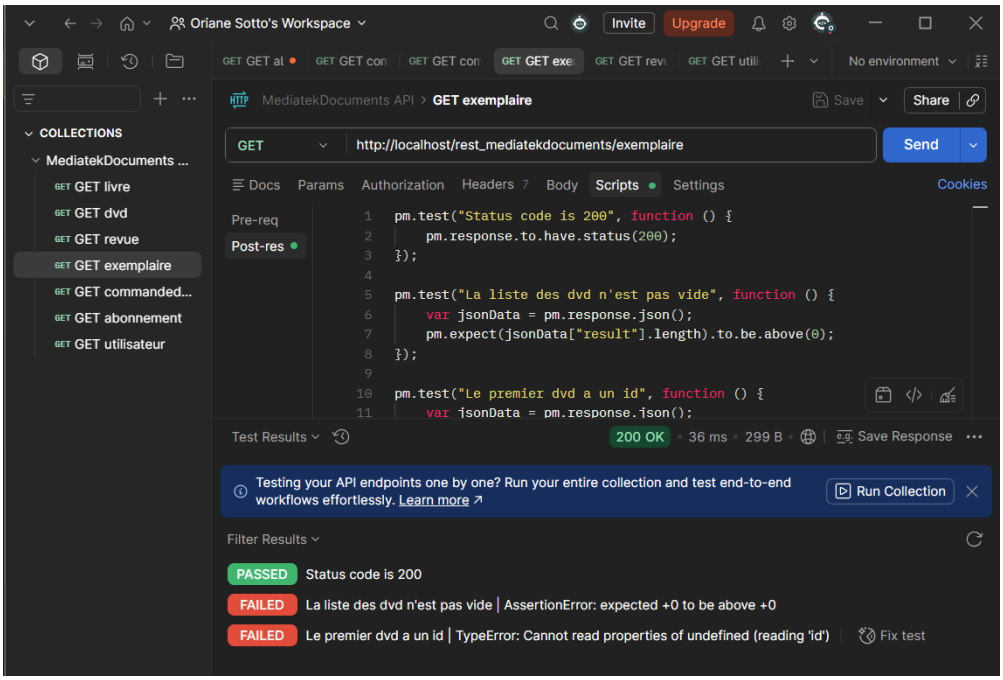


Capture Postman — GET revue : 3/3 PASSED

5.7 GET exemplaire

URL : http://localhost/rest_mediatekdocuments/exemplaire

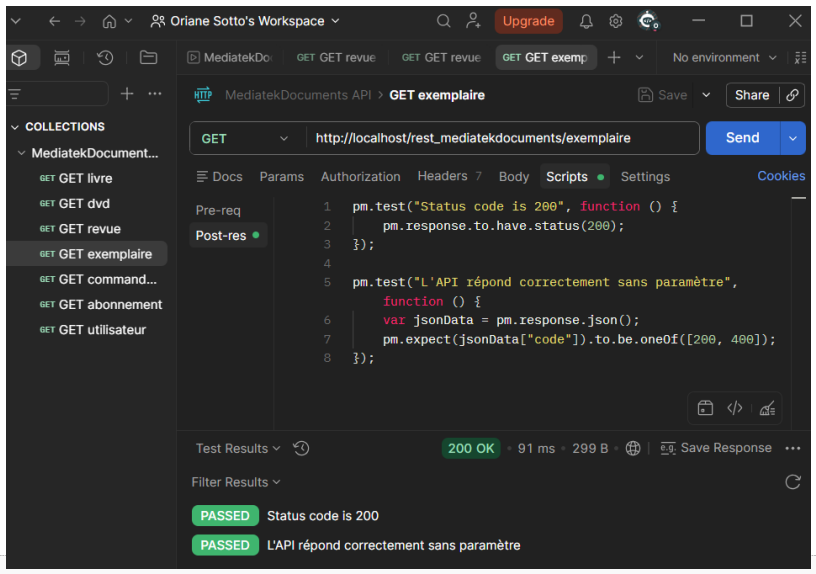
Cette route nécessite un paramètre id (identifiant d'une revue). Sans paramètre, l'API retourne une liste vide — le script initial identique à celui de livre a provoqué 2 erreurs.



Capture Postman — GET exemplaire AVANT correction : 1 PASSED + 2 FAILED (liste vide, id undefined)

Correction : le script a été adapté. Le test vérifie uniquement que l'API répond correctement, sans chercher de résultats.

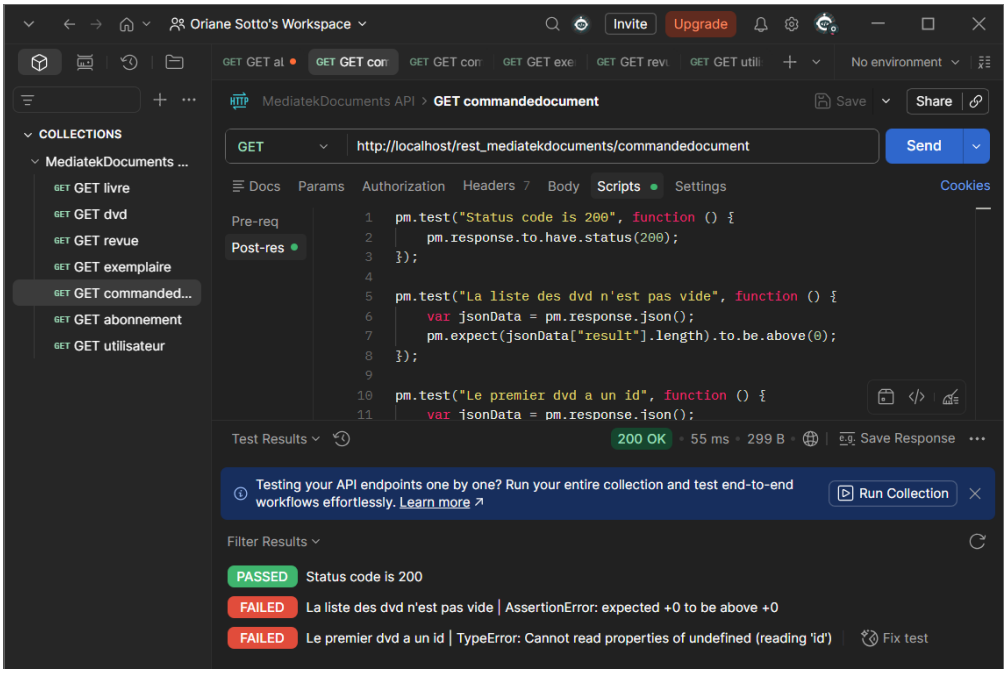
```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
pm.test("L'API répond correctement sans paramètre", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData["code"]).to.be.oneOf([200, 400]);
});
```



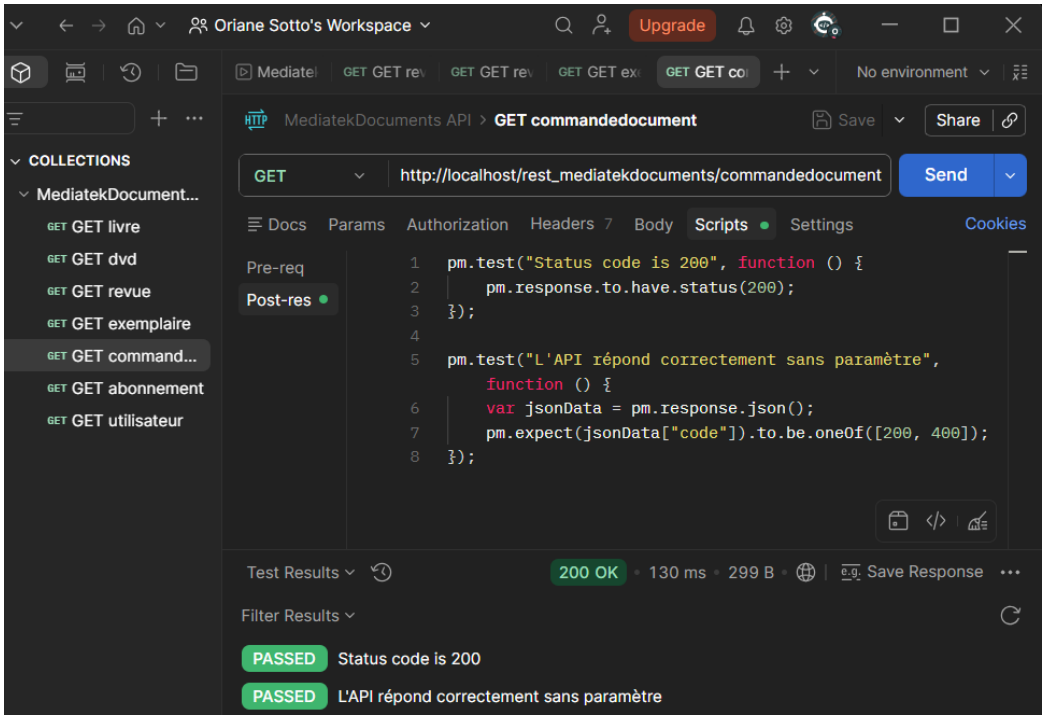
Capture Postman — GET exemplaire APRÈS correction : 2/2 PASSED

5.8 GET commandedocument

URL : http://localhost/rest_mediatekdocuments/commandedocument — même problème et même correction que exemplaire.



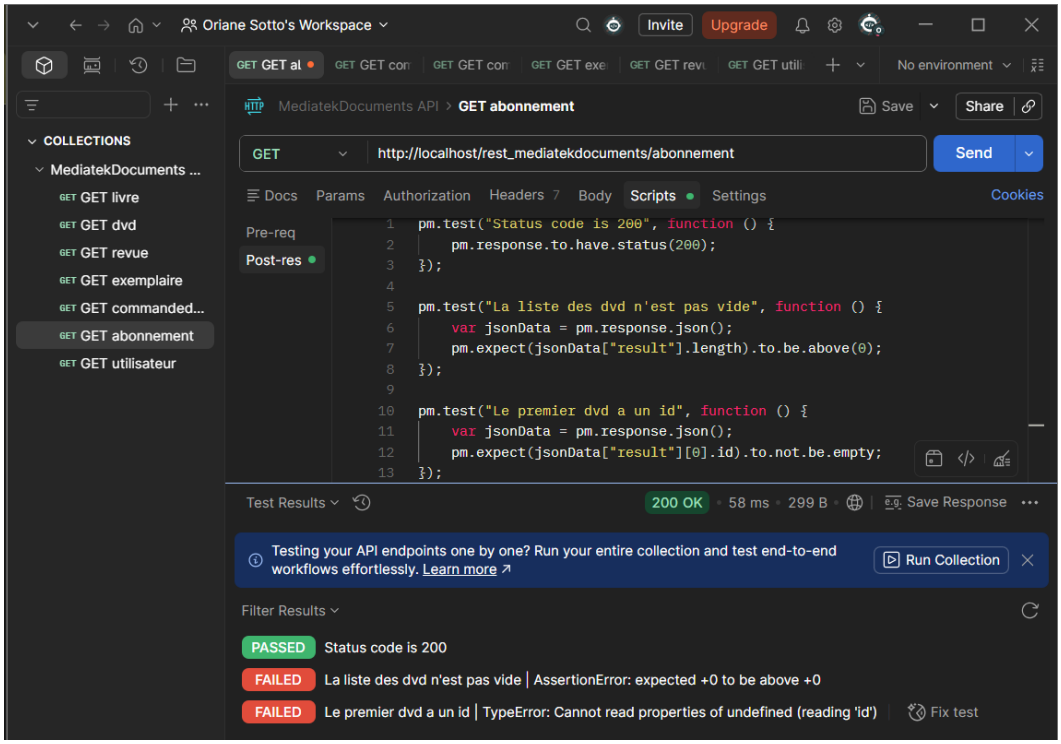
Capture Postman — GET commandedocument AVANT correction : 1 PASSED + 2 FAILED



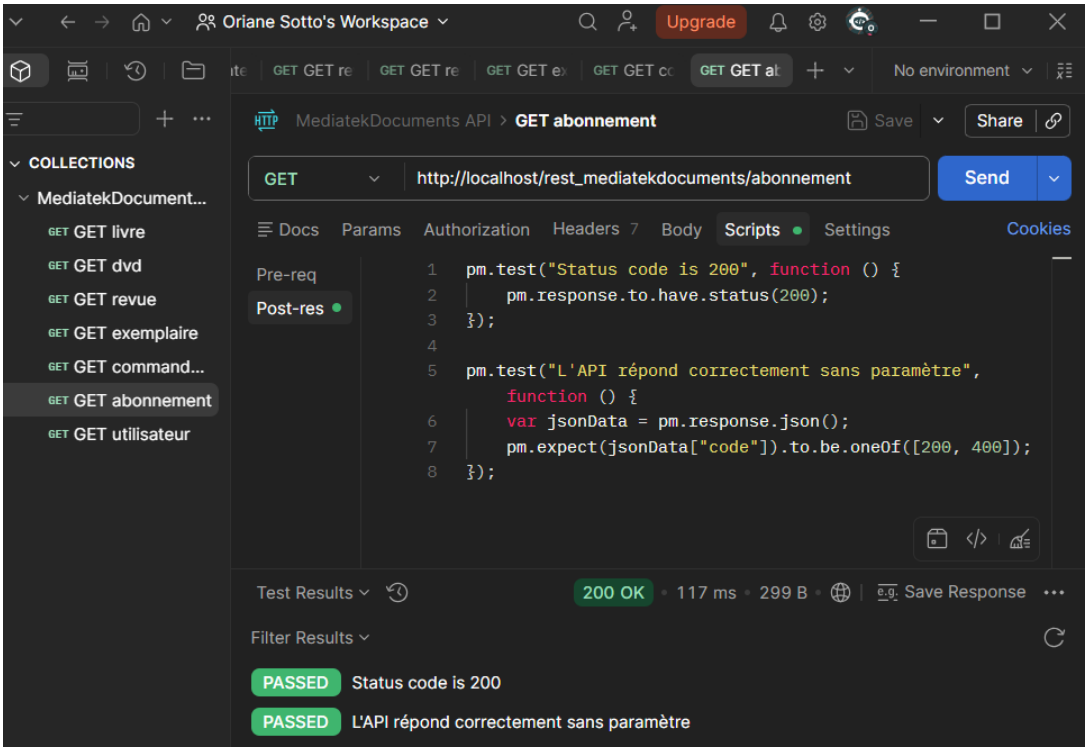
Capture Postman — GET commandedocument APRÈS correction : 2/2 PASSED

5.9 GET abonnement

URL : http://localhost/rest_mediatekdocuments/abonnement — même problème et même correction que exemplaire et commandedocument.



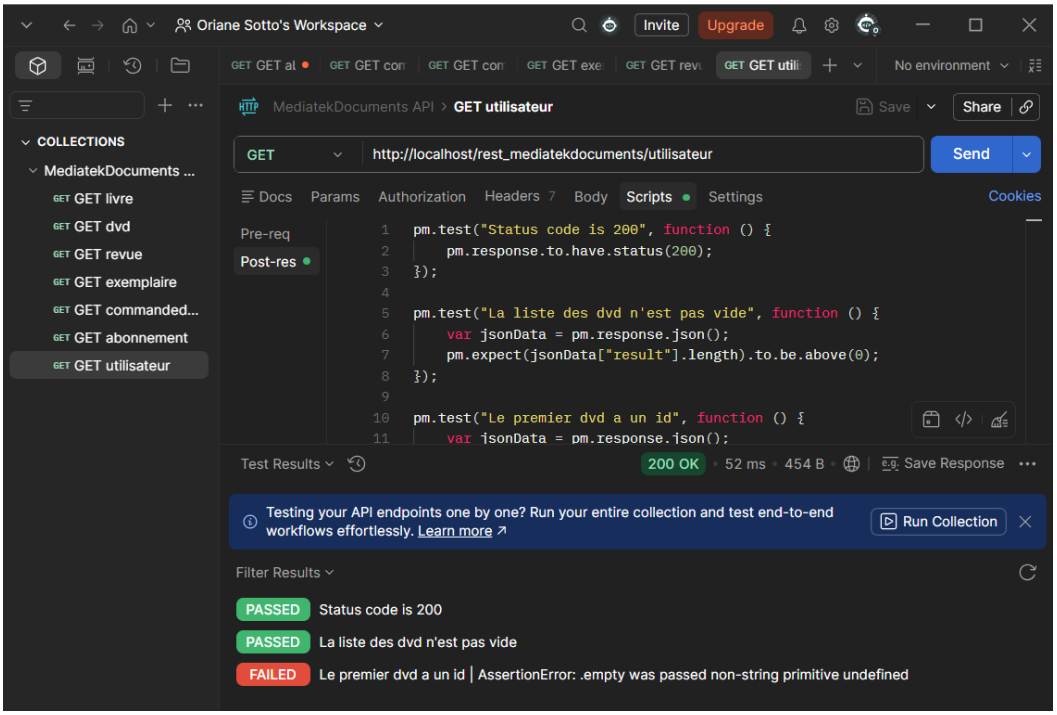
Capture Postman — GET abonnement AVANT correction : 1 PASSED + 2 FAILED

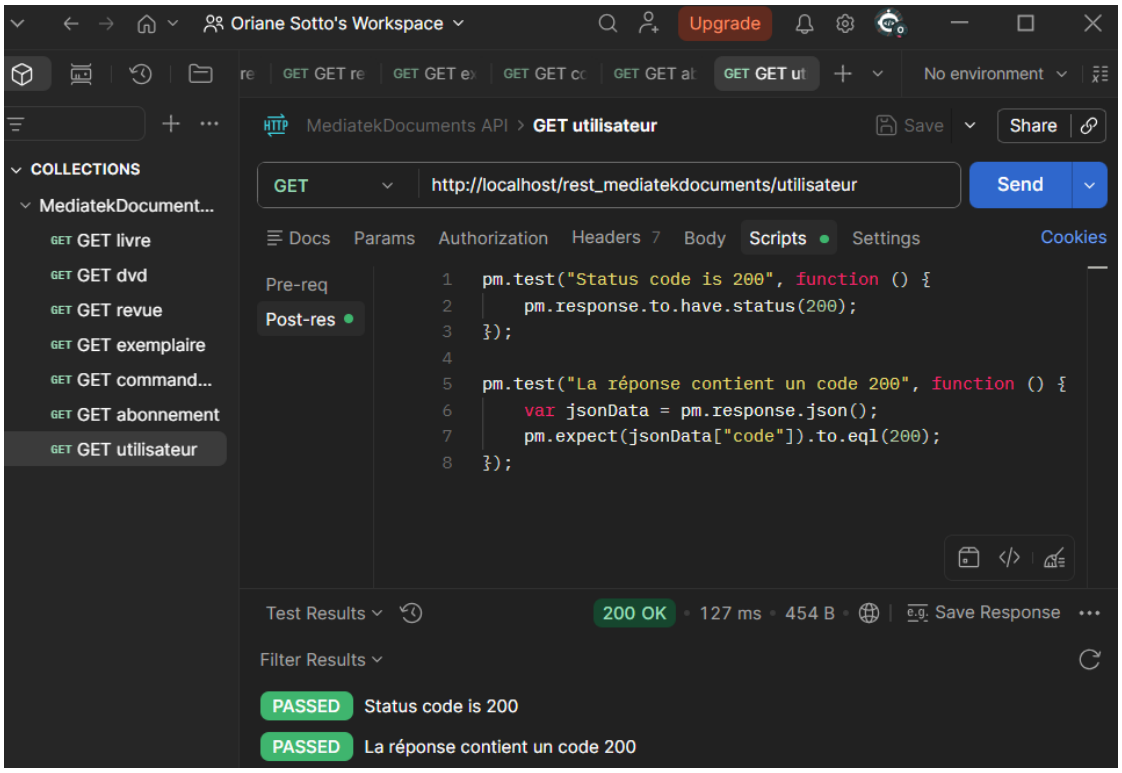


Capture Postman — GET abonnement APRÈS correction : 2/2 PASSED

5.10 GET utilisateur

URL : http://localhost/rest_mediatekdocuments/utilisateur — 2 tests : status 200 et code JSON retourné égal à 200.

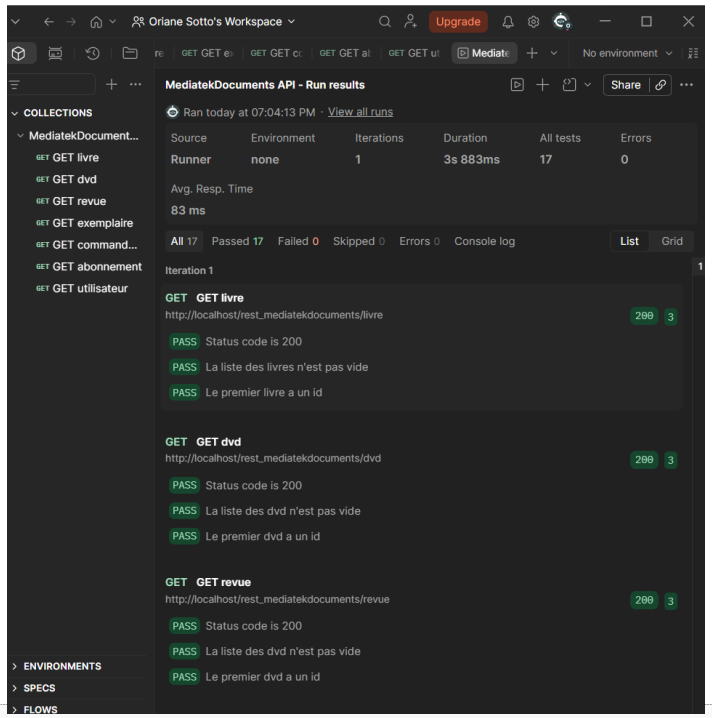




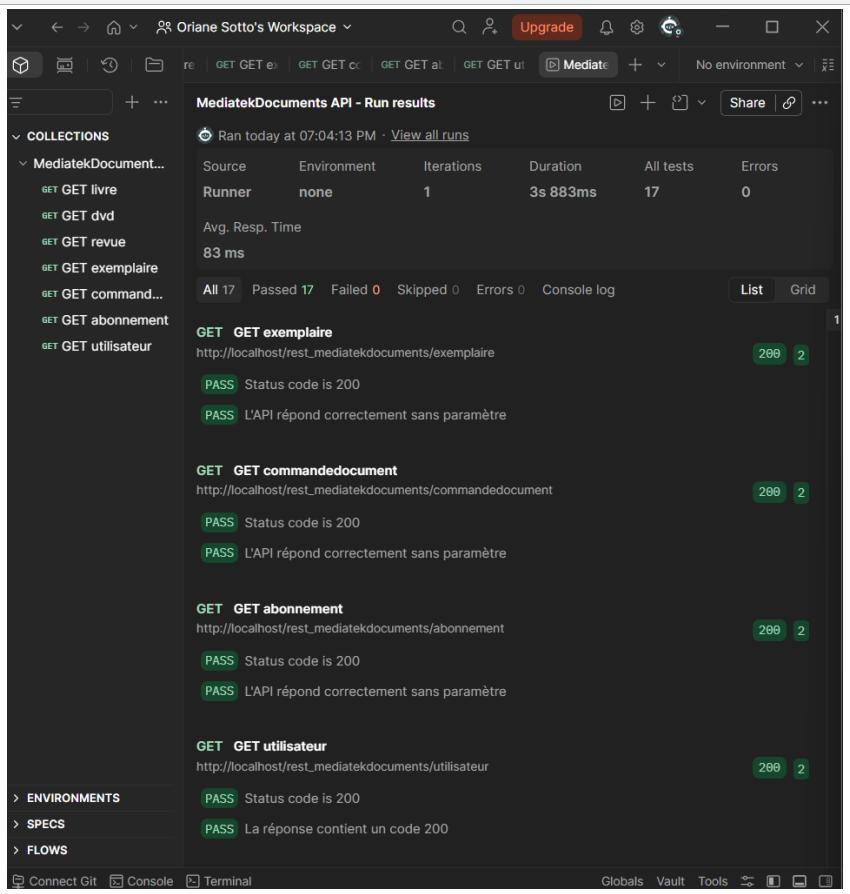
Capture Postman — GET utilisateur : 2/2 PASSED

6. Résultat global de la collection

Après correction de tous les scripts, la collection complète a été exécutée via Run Collection : 17 tests sur 17 réussis, 0 échec.



Capture Postman — Run Collection résultats haut : GET livre, dvd, revue PASSED



Capture Postman — Run Collection résultats bas : GET exemplaire, commandedocument, abonnement, utilisateur PASSED — 17/17 total

7. Plan de tests

Le plan de tests fourni par le CNED a été complété avec l'ensemble des tests effectués par catégorie.

Voir PDF

8. Résultat

L'ensemble des tests demandés ont été réalisés avec succès :

- 16 tests unitaires écrits et réussis sur toutes les classes du package Model
- 7 routes API testées dans Postman avec 17 tests au total — 17/17 PASSED après corrections
- Erreurs corrigées sur les routes nécessitant un paramètre (exemplaire, commandedocument, abonnement) — le comportement de retour de liste vide sans paramètre est normal et attendu
- Plan de tests complété